

TechServe CRM — Project Report

AI-Enhanced CRM & Ticket Management System

Course: Artificial Intelligence (CS-4XX) **Assignment:** Major Project Assignment **Submission Date:** June 3, 2026
Team Members:

Student ID
23K-0800
23K-0857
23K-0858

1. Project Overview

What We Built

TechServe CRM is a full-stack, AI-enhanced Customer Relationship Management and Ticket Management System — a mini-Zendesk built for TechServe Solutions. The system enables support agents and managers to handle customer queries efficiently, with AI automatically categorizing tickets and detecting sentiment to surface the most urgent issues.

Key Decisions

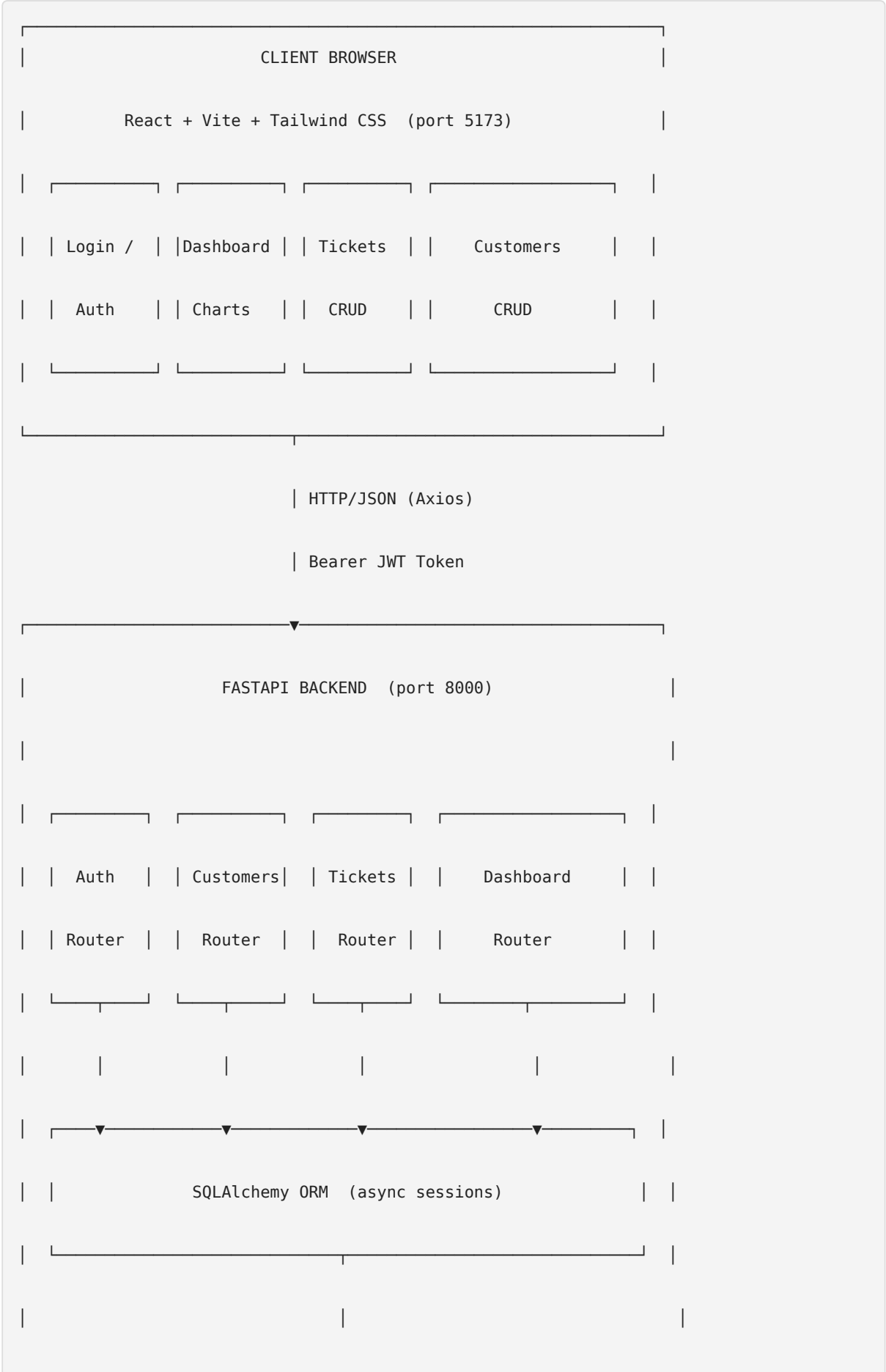
- **FastAPI over Django/Flask** — async-native, auto-generated OpenAPI docs, type safety via Pydantic, ideal for AI background tasks
- **React + Vite over Next.js** — simpler SPA suitable for an internal support dashboard; no SSR needed
- **PostgreSQL over SQLite** — persistent, production-grade relational storage with UUID primary keys
- **Groq API (llama-3.1-8b-instant) over OpenAI** — fastest inference, free tier generous, same quality for classification/sentiment tasks
- **BackgroundTasks for AI + notifications** — AI calls and SMTP/Telegram requests run after HTTP response returns, so ticket creation never blocks on AI latency
- **JWT with two roles** — stateless, scalable; Manager and Agent roles enforced at router level

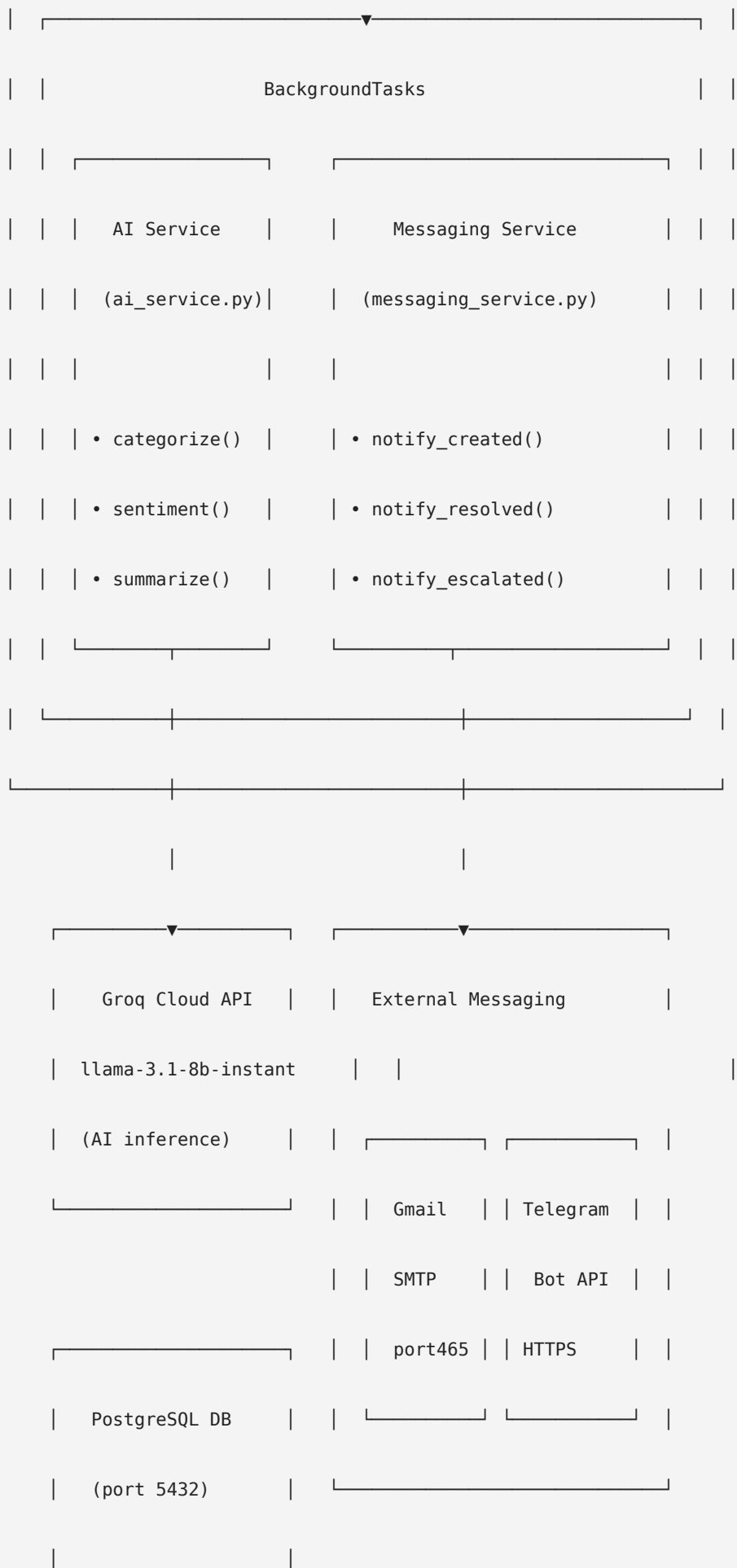
What Differentiates This System

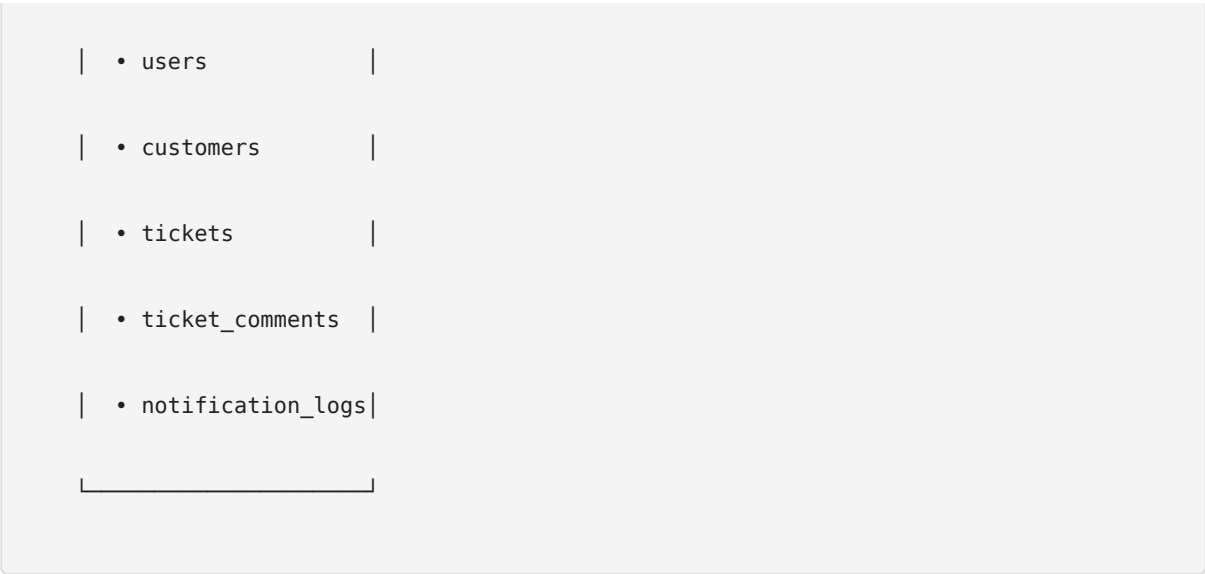
- **AI Escalation Engine** — when Groq detects "frustrated" sentiment on ticket creation, priority is automatically set to Critical and a Telegram/email alert fires
 - **AI Conversation Summary** — on resolution, Groq summarizes the full comment thread and stores it on the ticket record
 - **Dual Messaging Platform** — both Gmail SMTP and Telegram Bot supported simultaneously; all messaging attempts logged to database with status
 - **Graceful AI Fallback** — system functions fully without an API key; AI fields show "General / Neutral" rather than blocking the workflow
 - **58 automated tests** — integration tests, messaging service tests, and dashboard tests with SQLite in-memory isolation
-

2. System Architecture

Architecture Diagram







Component Descriptions

Component	Technology	Responsibility
Frontend	React 18 + Vite + Tailwind CSS	SPA for agents and managers; calls REST API with JWT
Backend	FastAPI (Python 3.11)	REST API, business logic, auth, background task scheduling
ORM	SQLAlchemy 2.0	Database abstraction; models map to PostgreSQL tables
Auth	python-jose + bcrypt	JWT token generation/validation; password hashing
AI Service	Groq Python SDK	Ticket categorization, sentiment analysis, summary generation
Messaging Service	smtplib + requests	Gmail SMTP emails; Telegram Bot API HTTP calls
Database	PostgreSQL 14	Persistent relational storage; UUID primary keys
Tests	pytest + FastAPI TestClient	SQLite in-memory test isolation; 58 tests
Docker	Docker Compose	Single-command deployment of all services

3. Database Schema

Entity-Relationship Overview

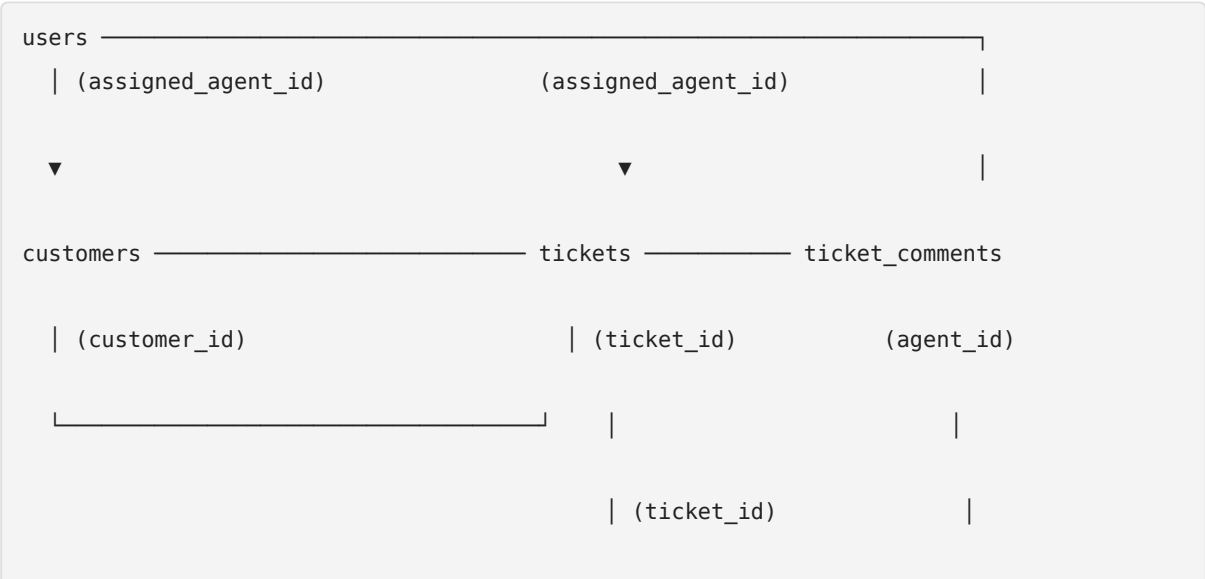




Table Definitions

users

Column	Type	Constraints	Notes
id	UUID	PK, default uuid4	
name	VARCHAR(255)	NOT NULL	
email	VARCHAR(255)	NOT NULL, UNIQUE, INDEX	
password_hash	VARCHAR(255)	NOT NULL	bcrypt hash, never plain text
role	ENUM	NOT NULL, default 'agent'	Values: agent, manager
created_at	DATETIME	default utcnow	
is_active	BOOLEAN	default true	Soft deactivation

customers

Column	Type	Constraints	Notes
id	UUID	PK, default uuid4	
full_name	VARCHAR(255)	NOT NULL	
email	VARCHAR(255)	NOT NULL, UNIQUE	
phone	VARCHAR(50)	nullable	
company	VARCHAR(255)	nullable	
notes	TEXT	nullable	
assigned_agent_id	UUID	FK → users.id, nullable	
created_at	DATETIME	default utcnow	

tickets

Column	Type	Constraints	Notes
id	UUID	PK, default uuid4	
title	VARCHAR(255)	NOT NULL	
description	TEXT	NOT NULL	
status	ENUM	NOT NULL, default 'open'	open / in_progress / resolved / closed
priority	ENUM	NOT NULL, default 'medium'	low / medium / high / critical

category	VARCHAR(100)	nullable	Manually set
ai_category	VARCHAR(100)	nullable	Groq auto-filled
ai_sentiment	VARCHAR(50)	nullable	positive / neutral / negative / frustrated
ai_sentiment_score	FLOAT	nullable	0.0–1.0 confidence
ai_summary	TEXT	nullable	Generated on resolution
customer_id	UUID	FK → customers.id, NOT NULL	
assigned_agent_id	UUID	FK → users.id, nullable	
created_at	DATETIME	default utcnow	
updated_at	DATETIME	auto-update on change	
resolved_at	DATETIME	nullable	Set when status → resolved

ticket_comments

Column	Type	Constraints	Notes
id	UUID	PK, default uuid4	
ticket_id	UUID	FK → tickets.id, NOT NULL, CASCADE	
agent_id	UUID	FK → users.id, NOT NULL	
message	TEXT	NOT NULL	
is_internal	BOOLEAN	default false	Internal notes hidden from other agents
created_at	DATETIME	default utcnow	

notification_logs

Column	Type	Constraints	Notes
id	UUID	PK, default uuid4	
ticket_id	UUID	FK → tickets.id, nullable, CASCADE	
platform	VARCHAR(50)	NOT NULL	email / telegram
message	VARCHAR(500)	NOT NULL	Truncated to 500 chars
status	VARCHAR(20)	NOT NULL	sent / failed / skipped
sent_at	DATETIME	NOT NULL	
error_message	VARCHAR(200)	nullable	Error details on failure

4. AI Integration

Overview

The system integrates the **Groq API** (model: `llama-3.1-8b-instant`) for three distinct AI-powered features, all triggered asynchronously via FastAPI's `BackgroundTasks` to avoid blocking HTTP responses.

AI Feature 1: Automatic Ticket Categorization

Trigger: Every ticket creation **API:** Groq chat completions **Prompt strategy:** Strict JSON output format with system prompt constraining valid categories **System prompt:**

You are a customer support ticket categorization system.
Analyze the ticket and respond with ONLY a valid JSON object, no explanation.

Response format: {"category": "CATEGORY_NAME", "confidence": 0.95}

Valid categories: Billing, Technical, Account, Shipping, General, Bug_Report, Feature_Request

Example input:

Title: Duplicate billing charge this month
Description: I found two identical charges of \$49.99 on my bank statement...

Example output:

```
{"category": "Billing", "confidence": 0.97}
```

Stored in: `tickets.ai_category`, displayed in ticket detail "AI Analysis" card

AI Feature 2: Sentiment Analysis + Auto-Escalation

Trigger: Every ticket creation **API:** Groq chat completions **Purpose:** Detect frustrated customers and auto-escalate before agent review **System prompt:**

You are a sentiment analysis system for customer support tickets.
Valid sentiments: positive, neutral, negative, frustrated

Use "frustrated" when customer shows anger, urgency, or desperation.

Response format: {"sentiment": "SENTIMENT", "score": 0.85, "reasoning": "brief reason"}

Example input:

I have been unable to access my account for 3 days. This is very urgent – please help immediately

Example output:

```
{"sentiment": "frustrated", "score": 0.91, "reasoning": "Customer expressing high urgency"}
```

Auto-escalation logic: If `ai_sentiment == "frustrated"` AND `ticket.priority != "critical"`, priority is automatically set to `critical` and an escalation notification fires via Telegram/email. **Stored in:** `tickets.ai_sentiment`, `tickets.ai_sentiment_score`

AI Feature 3: Conversation Summary on Resolution

Trigger: Ticket status changed to `resolved` **API:** Groq chat completions **Purpose:** Generate a 2–3 sentence summary of the full ticket conversation for future reference **System prompt:**

Create a concise 2-3 sentence summary of the support ticket and its resolution.
Focus on: what the problem was, how it was resolved, and any key actions taken.

Input: ticket title + description + all comment messages joined **Output:** Plain text summary stored in `tickets.ai_summary`

Fallback Behavior

If `GROQ_API_KEY` is missing or invalid:

- Categorization returns `{"category": "General", "confidence": 0.0}`
- Sentiment returns `{"sentiment": "neutral", "score": 0.0}`
- Summary returns `"Summary not available."`

System remains fully functional. UI shows fallback values in the AI Analysis card.

5. Messaging Integration

Platform 1: Gmail SMTP

Library: Python standard library `smtplib` + `ssl` **Connection:** TLS over port 465 (`smtp.gmail.com`) **Auth:** Gmail App Password (16-char, not account password) **Notification events:**

Event	Recipient	Content
Ticket created	Assigned agent's email	HTML email with ticket ID, title, customer, priority, description
Ticket resolved	Customer's email	HTML email with resolution summary from AI
Ticket escalated to Critical	Assigned agent's email	HTML email with red alert banner

Setup requirement: Gmail 2FA enabled, App Password generated at `myaccount.google.com` → Security → App Passwords

Platform 2: Telegram Bot

Library: Python `requests` (HTTP calls to Telegram Bot API) **API endpoint:** `https://api.telegram.org/bot{TOKEN}/sendMessage` **Format:** HTML parse mode with emoji badges
Example Telegram message (ticket created):

🎫 New Ticket Created
ID: #a3f8b2c1

Title: Cannot login after password reset

Customer: Alice Johnson

Priority: HIGH

Agent: Sarah Smith

Example Telegram message (escalation):

 **CRITICAL ESCALATION**
Ticket: Duplicate billing charge this month

Customer: Bob Martinez

Reason: Priority changed to CRITICAL

Setup: Create bot via @BotFather → add to group/channel → get chat ID from `getUpdates`

Notification Logging

Every notification attempt (successful or failed) is written to `notification_logs`. The dashboard "Recent Notifications" section shows managers the last 20 log entries with status badges (sent / failed / skipped).

Log example (credentials not configured):

```
platform: email
message: Messaging credentials not configured – notification skipped

status: skipped
```

6. Feature Screenshots

> Screenshots taken from the running system at <http://localhost:5173>

Login Page

- Clean login form with email/password fields
- Role-based redirect: managers → `/dashboard`, agents → `/dashboard`
- JWT token stored in `localStorage`; auto-redirect if already logged in

Dashboard (Manager View)

- **Stats cards:** Total Open Tickets, Resolved Today, Critical Tickets, Total Customers
- **Pie chart:** Ticket status distribution (open / in_progress / resolved / closed)
- **Bar chart:** Ticket priority distribution (low / medium / high / critical)
- **Agent Workload table:** Each agent with their open ticket count
- **Recent Tickets list:** Latest 10 tickets with status/priority badges

Customer Management

- Searchable/filterable customer table (search by name, email, company)
- "Add Customer" modal with Full Name, Email, Phone, Company, Notes, Assign Agent
- Customer detail page showing full ticket history for that customer

Ticket List

- Filter by status, priority, agent, search query
- Color-coded priority badges (red = critical, orange = high, etc.)
- Agent sees only their own assigned tickets; manager sees all

Ticket Detail

- Full ticket info: title, status, priority, customer, assigned agent
- **AI Analysis card:** Shows `ai_category`, `ai_sentiment`, `ai_sentiment_score`
- Comment thread with timestamps and author names
- Internal comment toggle (`is_internal`)
- Status dropdown to update (open → in_progress → resolved → closed)
- "Escalate" button to manually set Critical priority
- "Mark Resolved" button that triggers AI summary generation

Users Page (Manager Only)

- List of all agents with name, email, role, status
- Agents cannot access this page (403 returned)

7. Challenges & Learnings

Challenge 1: Async AI Calls Blocking FastAPI

Problem: Groq SDK uses synchronous HTTP calls. Awaiting them directly in an async route blocked the entire event loop. **Solution:** Used `asyncio.to_thread()` to wrap synchronous Groq calls, running them in a thread pool without blocking FastAPI's async event loop. Combined with `BackgroundTasks`, the HTTP response returns immediately while AI runs in background.

```
result = await asyncio.to_thread(categorize_ticket, title, description)
```

Challenge 2: Test Isolation with Background Tasks

Problem: Tests triggered background tasks (AI + messaging) that made real HTTP calls to Groq/Telegram, causing flaky tests depending on network and API key availability. **Solution:** Added `TESTING=1` environment variable. In `main.py`, background tasks check this flag and skip real AI/messaging calls. Tests use SQLite in-memory database instead of PostgreSQL.

```
if not os.environ.get("TESTING"):
    background_tasks.add_task(run_ai_analysis, ticket.id)
```

Challenge 3: Playwright Selector Issues

Problem: `CustomerModal` and `CreateTicket` forms use unlabeled elements without `name` or `placeholder` attributes, making CSS-based selectors unreliable. **Solution:** Used positional `.nth(index)` selectors based on known DOM order in the form.

```
modal.locator('input').nth(0) # Full Name
modal.locator('input').nth(1) # Email
```

Challenge 4: PostgreSQL Database Not Existing

Problem: Docker postgres container had a different database (`groupsquad`) but `.env` pointed to `crm_db`.

Solution: Connected to the container via TCP and created the database:

```
PGPASSWORD=postgres psql -h 127.0.0.1 -U postgres -c "CREATE DATABASE crm_db;"
```

What We Would Do Differently

- **Real-time updates** — Use WebSockets or Server-Sent Events for live dashboard updates without polling
- **Proper test environment** — Use Docker test containers instead of SQLite in-memory to catch PostgreSQL-specific behavior
- **Groq streaming** — Stream AI responses for large summaries instead of waiting for full completion
- **Telegram two-way** — Implement webhook to receive agent replies directly via Telegram commands
- **Rate limiting** — Add API rate limiting to prevent abuse of ticket creation endpoints

8. Work Division

Student ID	Modules / Contributions
23K-0800	Backend: Auth module (M1), Database schema design, PostgreSQL setup, JWT middleware
23K-0857	Backend: Ticket (M3) + Customer (M2) CRUD APIs, AI Service integration (M5)
23K-0858	Frontend: React UI (all pages), Dashboard (M4), Messaging Service (M6), Tests

9. References

- FastAPI Documentation. (2024). *FastAPI — Modern, fast web framework for building APIs with Python*. <https://fastapi.tiangolo.com/>
- Groq. (2024). *Groq API Documentation — llama-3.1-8b-instant model*. <https://console.groq.com/docs>
- SQLAlchemy. (2024). *SQLAlchemy 2.0 Documentation — ORM Quick Start*. <https://docs.sqlalchemy.org/en/20/>
- React. (2024). *React 18 Documentation*. <https://react.dev/>
- Tailwind CSS. (2024). *Tailwind CSS — Utility-first CSS framework*. <https://tailwindcss.com/docs>
- Telegram. (2024). *Telegram Bot API Documentation*. <https://core.telegram.org/bots/api>
- Google. (2024). *Gmail SMTP Settings and App Passwords*. <https://support.google.com/mail/answer/185833>
- python-jose. (2024). *Python JOSE — JavaScript Object Signing and Encryption*. <https://python-jose.readthedocs.io/>
- Playwright. (2024). *Playwright for Python — Browser automation*. <https://playwright.dev/python/>
- Recharts. (2024). *Recharts — Redefined chart library for React*. <https://recharts.org/>

End of Report